

```
List<Integer> numbers = Arrays.asList(10, 5, 3, 7, 2, 10, 5, 8, 9, 0, -3, 4);
List<String> names = Arrays.asList("Ali", "Mona", "Ahmed", "Sara", "Amr", "Laila",
"Kareem", "Nada", "Nour", "Samy", "", null);
```

1/ ♦ Basic Stream Operations

```
// Filter even numbers from a list of integers.
Predicate<Integer> ISEven=num->num%2==0;
List<Integer> noms=numbers.stream().filter(ISEven).collect(Collectors.toList());
//.....
```

```
//Find names starting with a specific letter from a list of strings.
Predicate<String> isNotNull = name -> name != null;
Predicate<String> startsWithA = name -> name.startsWith("A");
```

```
List<String> aNames = names.stream()
    .filter(isNotNull) // check if it is not null
    .filter(startsWithA)//start with A
    .collect(Collectors.toList());
//.....
```

```
//Convert all strings to uppercase using stream.
// 1. حروف كبيرة String وترجع String تعريف اللامبدا اللي بتاخذ 1.
Function<String, String> toUpperCaseFunc = name -> name.toUpperCase();
List<String> upperNames = names.stream()
    .filter(name -> name != null)
    .map(toUpperCaseFunc)
    .collect(Collectors.toList());
```

```
.....
//Sort a list of integers in descending order using streams
```

```
Comparator<Integer> descendingOrder = (n1, n2) -> n2.compareTo(n1);
List<Integer> sortedNumbers = numbers.stream() .sorted(descendingOrder)
.collect(Collectors.toList());
```

```
.....
//Remove duplicate elements from a list using distinct().
List<Integer> uniqueNumbers = numbers.stream()
    .distinct()
    .collect(Collectors.toList());
```

```
*****
```

2/ ♦ Intermediate Stream Tasks

```
//Count the number of strings longer than 5 characters
long longNamesCount = names.stream()
```

```
.filter(name -> name != null && name.length() > 5)
.count();
```

```
.....
//Find the first element in a stream that matches a given condition
```

الفكرة: عايزين أول عنصر يحقق شرط معين (مثلاً رقم أكبر من 7). بنستخدم **findFirst** ودي بترجع **Optional** (عشان ممكن مفيش حاجة تحقق الشرط).

```
Optional<Integer> firstMatch = numbers.stream()
    .filter(n -> n > 7)
    .findFirst();
```

```
.....
//Check if any number is divisible by 5 in a list
```

الفكرة: هل "يوجد على الأقل" رقم يقبل القسمة على 5؟ دالة **anyMatch** بترجع **true** أو **false**

```
boolean isDivisibleBy5 = numbers.stream()
    .anyMatch(n -> n % 5 == 0);
```

```
.....
//Collect elements into a Set instead of a List
```

```
Set<Integer> numbersSet = numbers.stream()
    .collect(Collectors.toSet());
```

```
.....
//Skip the first 3 elements and return the rest
```

الفكرة: نتجاهل أول 3 عناصر وناخد الباقي. دالة **skip(n)** بتعمل كذا بالظبط.

```
List<Integer> skippedNumbers = numbers.stream()
    .skip(3)
    .collect(Collectors.toList());
```

3/ ♦ Numeric Streams & Reductions

```
//Calculate the sum of a list of integers using reduce
```

الفكرة: **reduce** بتاخذ القيم كلها وتدمجهم في قيمة واحدة بناءً على عملية (هنا الجمع).

```
int sum = numbers.stream()
    .reduce(0, Integer::sum); // هي القيمة الابتدائية 0
```

```
.....
//Find the maximum and minimum value in a list
```

الفكرة: بنستخدم **max** و **min** وبنديهم طريقة المقارنة

```
Optional<Integer> maxNum = numbers.stream().max(Integer::compareTo);
Optional<Integer> minNum = numbers.stream().min(Integer::compareTo);
```

```
.....
//Calculate the average of a list of doubles
```

الفكرة: الجافا فيها Stream مخصص للأرقام زي `DoubleStream`. بنحول العناصر لأرقام بـ `mapToDouble` ونجيب الـ `average`.

```
OptionalDouble avgGrade = students.stream()
    .mapToDouble(Student::getGrade)
    .average();
.....
//Multiply all integers in a list together using reduce
```

الفكرة: زي الجمع، بس العملية هتكون ضرب، والقيمة الابتدائية هتكون 1 (عشان الضرب في 0 بيصفر النتيجة).

```
int multiplication = numbers.stream()
    .reduce(1, (a, b) -> a * b);
.....
//Count how many numbers are positive in a list
```

الفكرة: فلتر الأرقام الأكبر من صفر، وبعدين عددهم.

```
long positiveCount = numbers.stream()
    .filter(n -> n > 0)
    .count();
*****
```

4/ ♦ Collectors & Grouping

```
//Group a list of students by their department
```

الفكرة: عايزين نقسم الطلاب في "خريطة" (Map)، الـ Key هو القسم، والـ Value هي قائمة الطلاب اللي في القسم ده.

```
Map<String, List<Student>> studentsByDept = students.stream()
    .collect(Collectors.groupingBy(Student::getDepartment));
.....
```

```
//Partition a list of numbers into even and odd
```

الفكرة: الـ `partitioningBy` بتقسم الداتا لمجموعتين بس (`true` للزوجي و `false` للفردى) بناءً على شرط

```
Map<Boolean, List<Integer>> evenOddPartition = numbers.stream()
    .collect(Collectors.partitioningBy(n -> n % 2 == 0));
.....
```

```
//Create a comma-separated string from a list of strings
```

الفكرة: عايزين نلزنق الأسماء جنب بعض وبينهم فاصلة ،. بنستخدم `joining`.

```
String commaSeparatedNames = names.stream()
    .filter(name -> name != null && !name.isEmpty())
    .collect(Collectors.joining(", "));
.....
```

```
//Group employees by age and count how many per age
```

الفكرة: نجمع الموظفين بالعمر، بس بدل ما نجيب ليستة الموظفين، احنا عايزين "عددهم" بس

```
Map<Integer, Long> countByAge = employees.stream()
    .collect(Collectors.groupingBy(Employee::getAge, Collectors.counting()));
```

```
.....
//Find the average salary per department
```

الفكرة: نجمع بالقسم، ونحسب متوسط (**averagingDouble**) الرواتب لكل قسم

```
Map<String, Double> avgSalaryByDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment,
    Collectors.averagingDouble(Employee::getSalary)));
```

5/ ♦ Optional, Map, FlatMap

//Flatten a list of lists into a single list

الفكرة: عندك List جواها Lists (زي **nestedWords**). عايزين نخليهم كلهم في List واحدة سطحية. **flatMap** بتفك القوائم دي وتدمجهم.

```
List<String> flatWords = nestedWords.stream()
    .flatMap(List::stream)
    .collect(Collectors.toList());
```

```
.....
//Extract all unique characters from a list of words
```

الفكرة: هنفك الكلمات لحروف، ونجمع الحروف دي بدون تكرار.

```
List<String> uniqueChars = nestedWords.stream()
    .flatMap(List::stream) // فكينا القوائم لكلمات
    .flatMap(word -> Arrays.stream(word.split(""))) // فكينا الكلمات لحروف
    .distinct()
    .collect(Collectors.toList());
```

```
.....
//Filter a list of Optionals and collect non-empty values
```

الفكرة: لو عندك ليستة **Optional**، وعايز تاخذ القيم اللي موجودة بس (مش Empty).

```
// List<String> presentValues = optionals.stream()
//     .flatMap(Optional::stream) // دي بتطلع القيمة لو موجودة وتتجاهلها لو فاضية
//     .collect(Collectors.toList());
```

```
.....
//Map a list of strings to their lengths
```

الفكرة: نحول من **String** إلى **Integer** (طول الكلمة).

```
List<Integer> nameLengths = names.stream()
```

```
.filter(Objects::nonNull)
.map(String::length)
.collect(Collectors.toList());
```

```
.....
//Return a list of uppercased words that start with "A"
```

الفكرة: فلتر الأول (يبدأ بـ A) بعدين حول (كابيتال). الترتيب مهم بشأن الأداء!

```
List<String> upperA_Names = names.stream()
    .filter(name -> name != null && name.startsWith("A"))
    .map(String::toUpperCase)
    .collect(Collectors.toList());
```

6/ ♦ Advanced Operations

//Sort a list of employees by salary then by name

الفكرة: الترتيب المتعدد. بنرتب بالراتب الأول، ولو تساوا (thenComparing) بنرتب بالاسم.

```
List<Employee> sortedEmployees = employees.stream()
    .sorted(Comparator.comparing(Employee::getSalary).thenComparing(Employee::getName))
    .collect(Collectors.toList());
```

```
.....
//Find the second highest number in a list
```

الفكرة: شيل المتكرر -> رتب تنازلي -> تجاهل أول رقم (skip(1)) -> هات اللي عليه الدور (findFirst).

```
Optional<Integer> secondHighest = numbers.stream()
    .distinct()
    .sorted(Comparator.reverseOrder())
    .skip(1)
    .findFirst();
```

```
.....
//Remove null or empty strings from a list
```

الفكرة: تأمين الداتا. بنشيل الـ null والمسافات الفاضية

```
List<String> validNames = names.stream()
    .filter(name -> name != null && !name.trim().isEmpty())
    .collect(Collectors.toList());
```

```
.....
//Partition students into pass/fail groups based on grade
```

الفكرة: تقسيم لنصين (ناجح وساقط) بناءً على شرط (الدرجة <= 50).

```
Map<Boolean, List<Student>> passFailStudents = students.stream()
    .collect(Collectors.partitioningBy(s -> s.getGrade() >= 50));
// true -> الناجحين , false -> الساقطين
```

